

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART XIII

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I:	Overview of Agentic Coding Assistants
Part – II:	Claude Ecosystem, Installing & using Claude Code
Part – III:	Slash Commands, Context Window & `.claude` Directory
Part – IV:	Workflow of Building a Basic Vulnerable Web Application
Part – V:	Exploiting Vulnerable Web Application
Part – VI:	Workflow of Implementing Features and Fixing Bugs
Part – VII:	Collaborative Workflow: Claude Code + GitHub + Jira
Part – VIII:	Built-in Skills & Vuln-Web-App Current Status
Part – IX:	Creating & using Custom Skills & Skill Marketplace
Part – X:	Deep Dive: Conceptual Understanding of MCP
Part – XI:	MCP in Action
Part – XII:	**Creating / Using Built-in & Custom Subagents**
Part – XIII:	**Git Worktrees, Agent Teams & a revisit to OWASP Top 10**

Multi-Agent Architecture

Choosing the Right Multi-Agent Architecture

Decision Framework

Start with the simplest solution that satisfies your workflow.
Only add more coordination or parallelism when the task requires it.

1. **Do you need help with one focused task?**
   - YES → **Subagent**
     Focused reasoning
     Single task, isolated context.
   - NO ↓
2. **Can these tasks run independently in parallel?**
   - YES → **Background Agents**
     Parallel Claude sessions
     Independent, separate work.
   - NO ↓
3. **Do the tasks need isolated file edits?**
   - YES → **Git Worktrees**
     Independent working directories
     Parallel code, no conflicts.
   - NO ↓
4. **Must the AI workers collaborate on tasks?**
   - YES → **Agent Team**
     Collaborative AI workers
     Shared task list & messaging.
   - NO ↓

✓ No further branching required.

Architecture Summary

Subagent
- One focused task
- One session
- Lowest coordination



Background Agents
- Independent sessions
- Parallel execution
- Human supervision



Git Worktrees
- Separate working directories
- Parallel code edits
- Manual coordination



Agent Team
- Shared task list
- Team mailbox
- Collaborative execution



Recommendation

Choose the simplest architecture that solves the problem.

Begin with a Subagent for focused work. Move to Background Agents for independent parallel execution, Git Worktrees when isolated code changes are required, and Agent Teams only when multiple AI workers must communicate and coordinate on the same project.

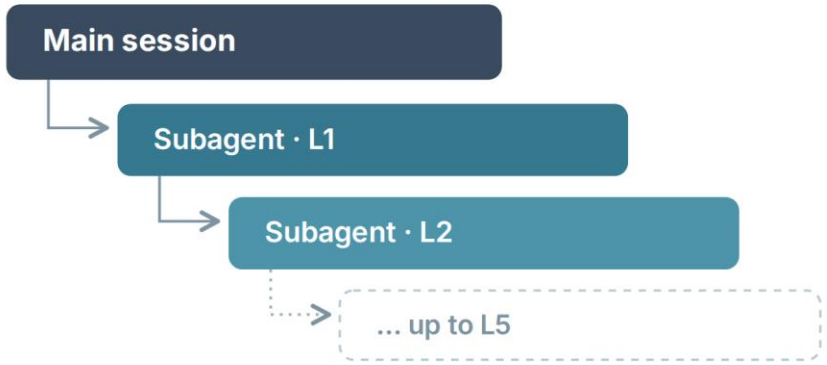
How Deep Does Delegation Go?

How Deep Does Delegation Go?

DELEGATION DEPTH — Two different answers. **Subagents** now nest into a hierarchy; **agent teams** stay flat.

Subagents — a hierarchy NEW

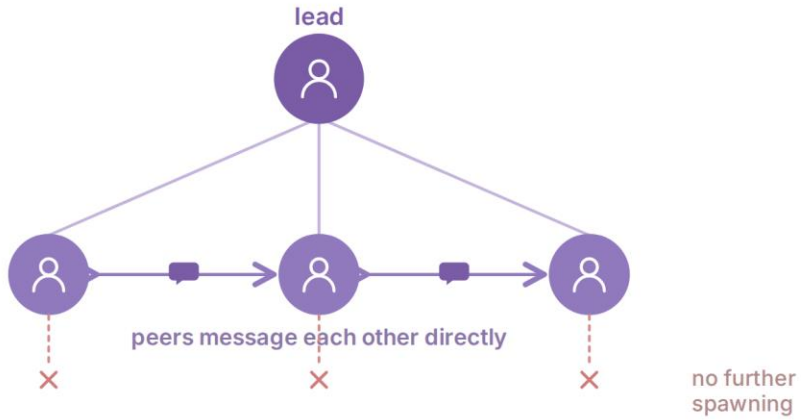
each hands work further down · own context each



Up to 5 levels deep (v2.1.172) — the depth limit is fixed, and only the top subagent's summary returns to you.

Agent teams — flat

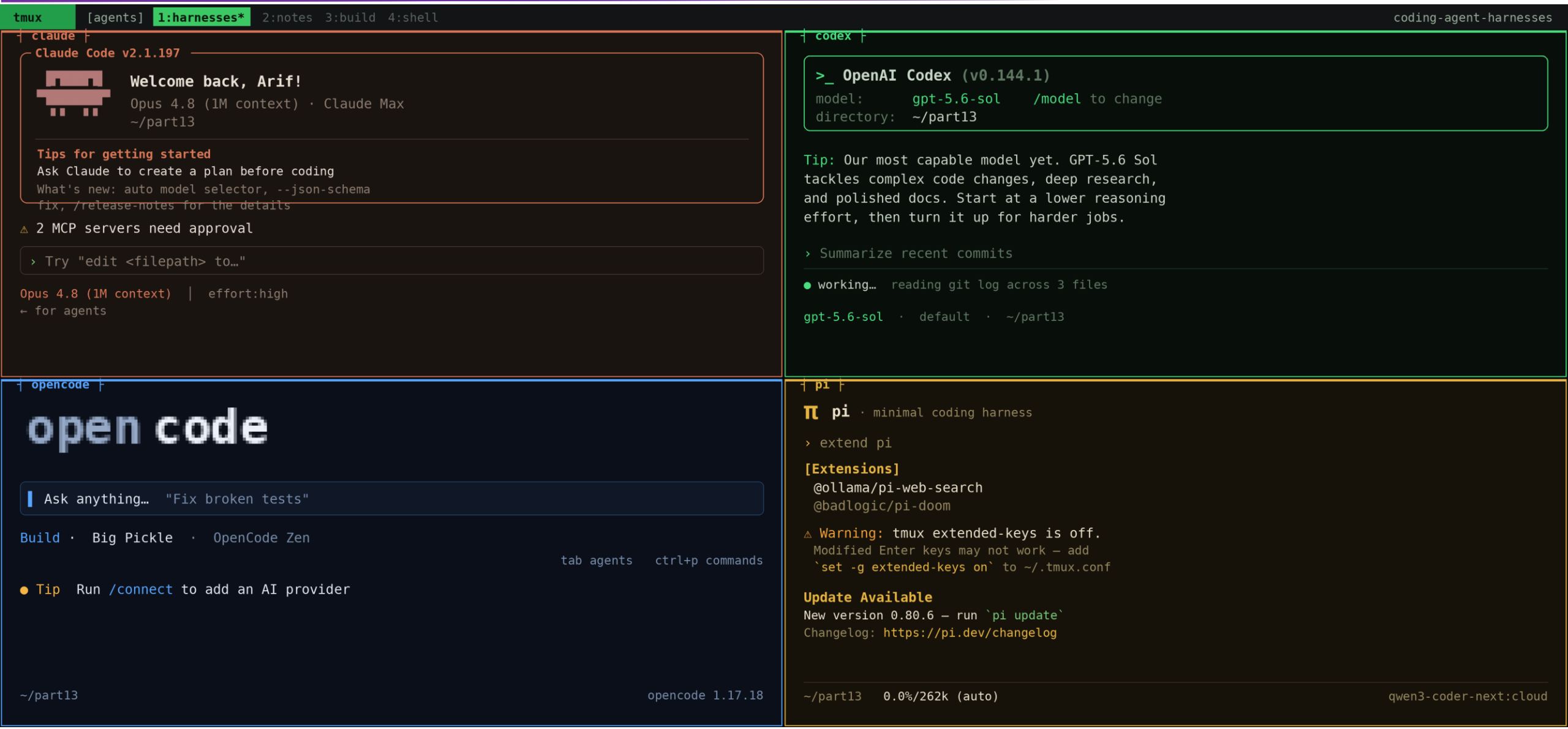
teammates are peers · one team per session



No nesting — teammates can't spawn teammates. The lead assigns and synthesizes, but teammates **message each other directly**.

Rule of Thumb | **Nest subagents** to push noise deeper out of your context. **Reach for a team** when the workers must talk to each other.

Multiple Agent Harnesses in tmux



Recap: Agent Memory

Agent Memory, Mapped to Claude Code

Four memory types from the CoALA framework (Princeton, 2023) — abstract overview

1 WORKING MEMORY

Short-term · active workspace

IN CLAUDE CODE
The live context window

HOLDS
Current task, instructions, tool results, running conversation

FED BY
RAG / retrieval, MCP tool calls, on-demand file reads

MANAGED BY
Context editing, tool-result clearing, compaction

WATCH OUT
As it fills, stale context degrades attention and answers

LIFETIME
Single session · volatile

2 SEMANTIC MEMORY

Facts & knowledge

IN CLAUDE CODE
CLAUDE.md (+ .claude/rules)

HOLDS
Conventions, build commands, architecture, “always do X” rules

SCOPES
managed → project → user → local (concatenated, additive)

NATURE
Advisory context loaded as a user message — not enforced config

EDIT VIA
/memory or edit the file directly; keep it short and stable

LIFETIME
Persistent · human-authored · version-controlled

3 PROCEDURAL MEMORY

Skills · “knowing how”

IN CLAUDE CODE
Skills (SKILL.md)

HOLDS
Reusable workflows, step-by-step procedures, bundled scripts

STRUCTURE
SKILL.md + scripts/ + references/ + assets/

MECHANISM
Progressive disclosure: discovery → activation → execution

EFFICIENT
Only name + description at startup; full body loads on trigger

LIFETIME
Persistent · on-demand · portable open standard

4 EPISODIC MEMORY

Past events & experience

IN CLAUDE CODE
Auto memory

HOLDS
Notes Claude writes itself — debugging insights, preferences, habits

WHERE
~/.claude/projects/<project>/memory/ · MEMORY.md index

MECHANISM
Auto-captured, then consolidated (Dreams merges dupes, drops stale)

DEFAULT
On in v2.1.59+ · toggle with /memory

LIFETIME
Persistent across sessions · per-repository

Sources: Anthropic Claude Code docs · Agent Skills · CoALA (arXiv:2309.02427)

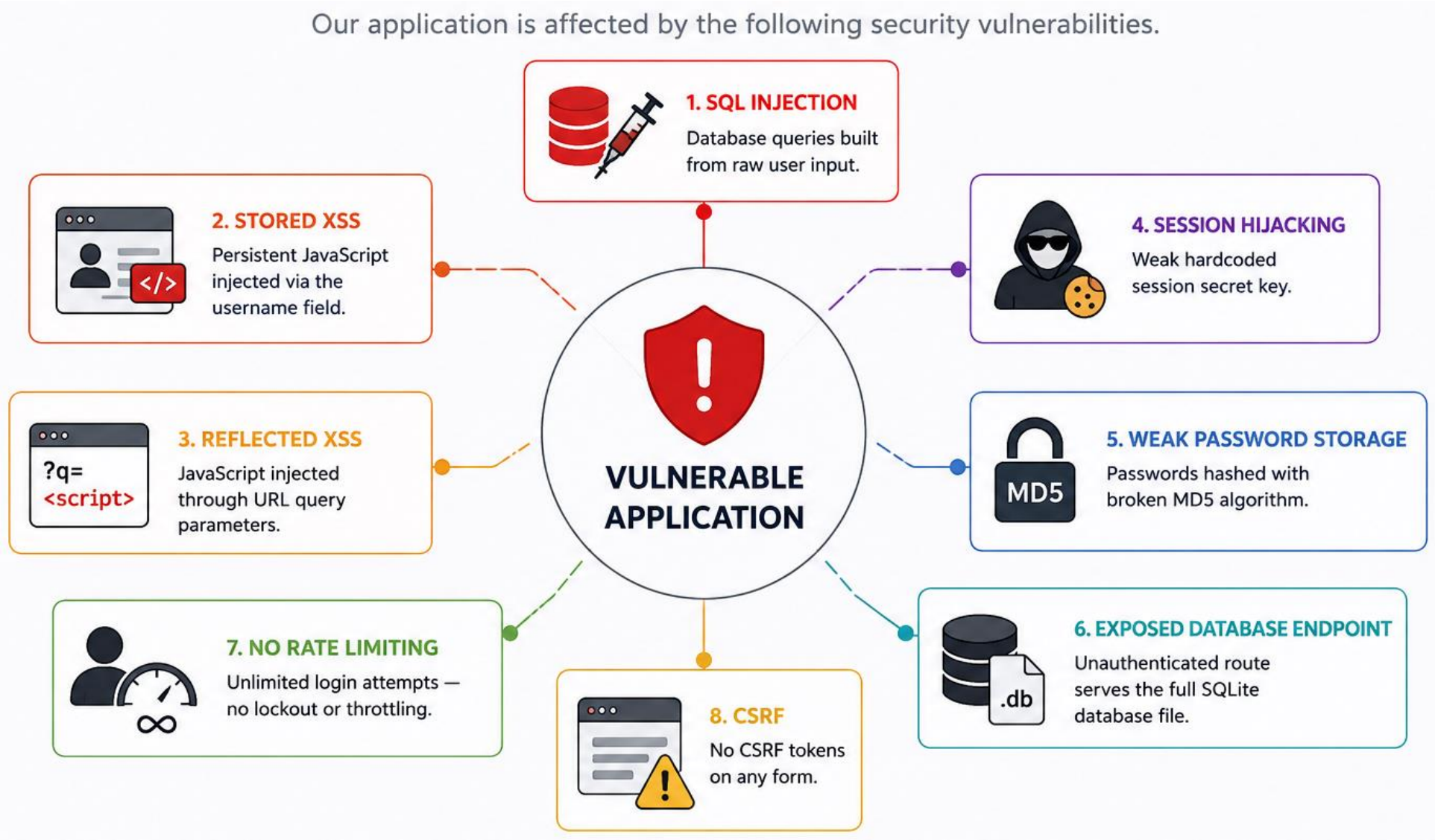
Working · Semantic · Procedural · Episodic



Git Worktrees

Instructor: Muhammad Arif Butt, PhD

Vulnerable Web App

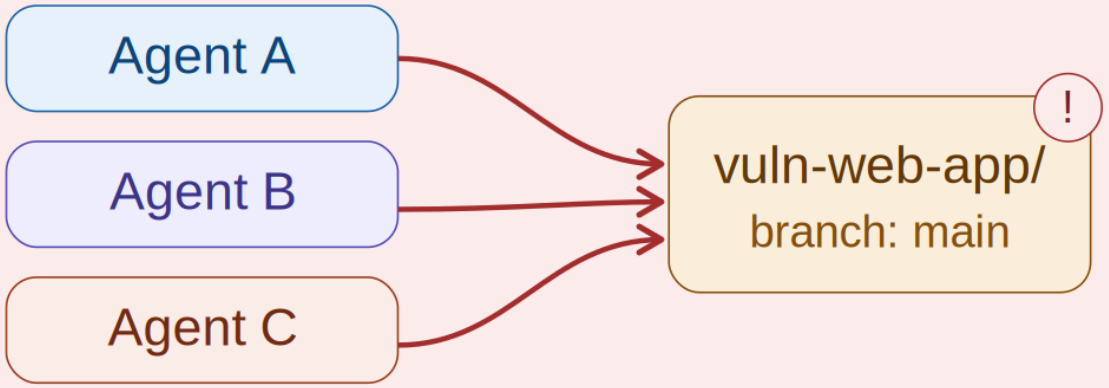


Why git Worktrees?

Three agents, one folder — why git worktrees

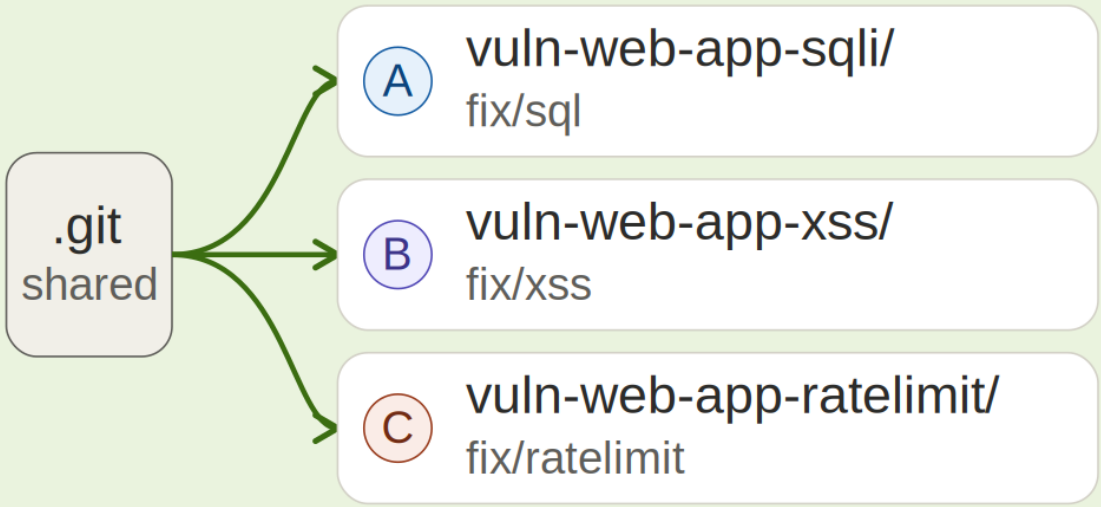
Three parallel Claude sessions fixing three vulnerabilities in the vuln-web-app repo

The problem



One folder, one checkout — agents overwrite each other

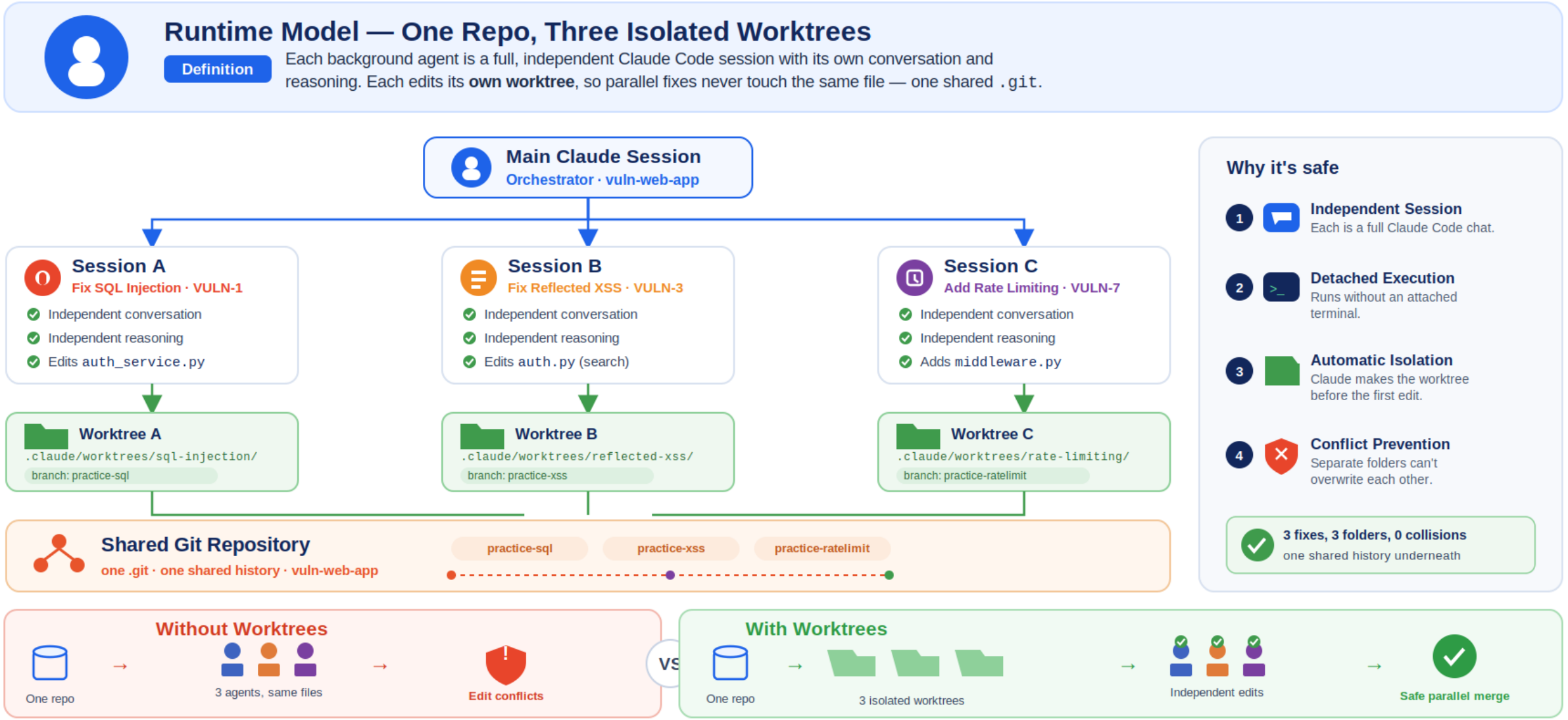
The solution: git worktree



What is a git worktree?

A second working copy of your repository — its own folder, on its own branch, sharing one .git history.
Only the files are copied, never the history, so each worktree is cheap to create and to throw away.

Runtime Model: One Repo - Three Worktrees



Start Claude in a Worktree

Start Claude in a Worktree — One Command

No manual git juggling. **`claude -w <name>`** makes an isolated copy, creates a branch, and drops Claude inside it. Open two more terminals, run it again — now **three Claudes work in parallel.**

Terminal 1 — in vuln-web-app/

```
$ claude -w sql-injection

☐ created .claude/worktrees/sql-injection/
☐ on new branch   worktree-sql-injection
☐ copied ignored files (.env)

☐ Claude ready — working in the worktree
> fix the SQL injection in auth_service.py
```

Terminal 2 — in vuln-web-app/

```
$ claude -w reflected-xss

☐ created .claude/worktrees/reflected-xss/
☐ on new branch   worktree-reflected-xss
☐ copied ignored files (.env)

☐ a second Claude, fully isolated
> fix the reflected XSS in the search route
```

Terminal 3 — in vuln-web-app/

```
$ claude -w rate-limiting

☐ created .claude/worktrees/rate-limiting/
☐ on new branch   worktree-rate-limiting
☐ copied ignored files (.env)

☐ a third Claude, fully isolated
> add rate limiting in middleware.py
```

What each command did

- **Made an isolated copy** — under `.claude/worktrees/`
- **Created a new branch** — `worktree-<name>`
- **Opened a session there** — your main folder untouched
- **Three Claudes, in parallel** — separate folders, no clashes

Worktrees live under .claude/worktrees/<name>/ at the repo root.

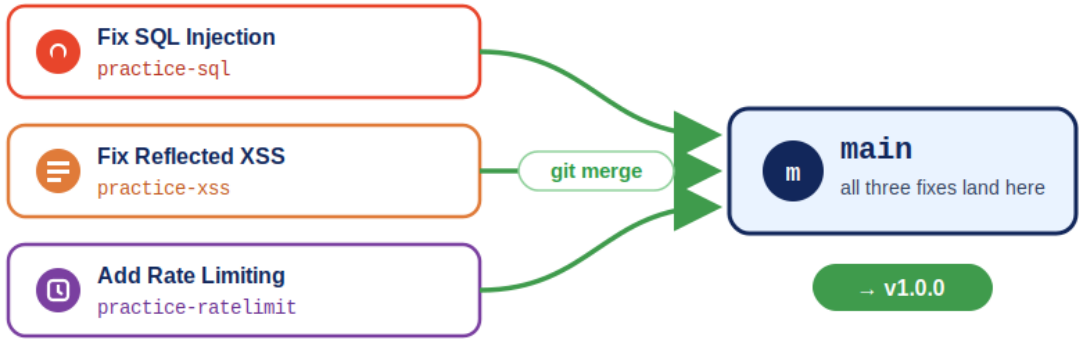
Tip: add **`.claude/worktrees/`** to **`.gitignore`** so these copies don't clutter **`git status`**.

Merge the Work

Merge the Work Back

Each worktree is on a **normal git branch**. Bring them into **`main`** one at a time — ask Claude, or run **`git merge`** yourself. Merge one branch, resolve if needed, then the next.

Three branches, merged one at a time



Ask Claude — plain English

```
> merge practice-sql into main
▫ Merged. 3 files changed.
```

...or run it yourself

```
$ git merge practice-sql
Fast-forward · commit — done
```

If it says “conflict”

1. Git names the file that clashed
2. Open it, keep the right lines, delete the **<<<<** markers
3. Commit — done

In this example the three fixes touch different files, so conflicts are unlikely — but the branches are ordinary git either way.

Merge one branch at a time — that's the whole workflow.

Three isolated fixes flow back into **`main`**, turning **v0.1.1 → v1.0.0** in parallel instead of one after another.

Hands-on Demo: Git Worktrees



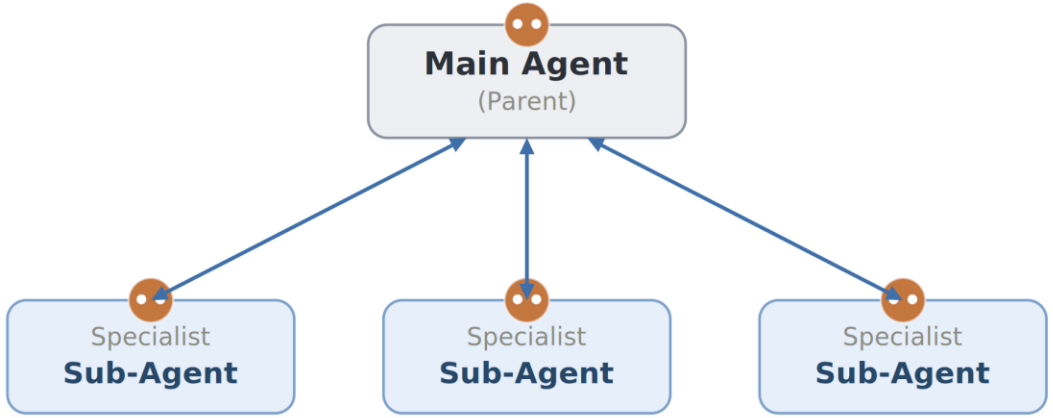
Agent Teams

Instructor: Muhammad Arif Butt, PhD

The Problem: Subagents Can't Talk

Sub-Agents

Delegation & Bottleneck

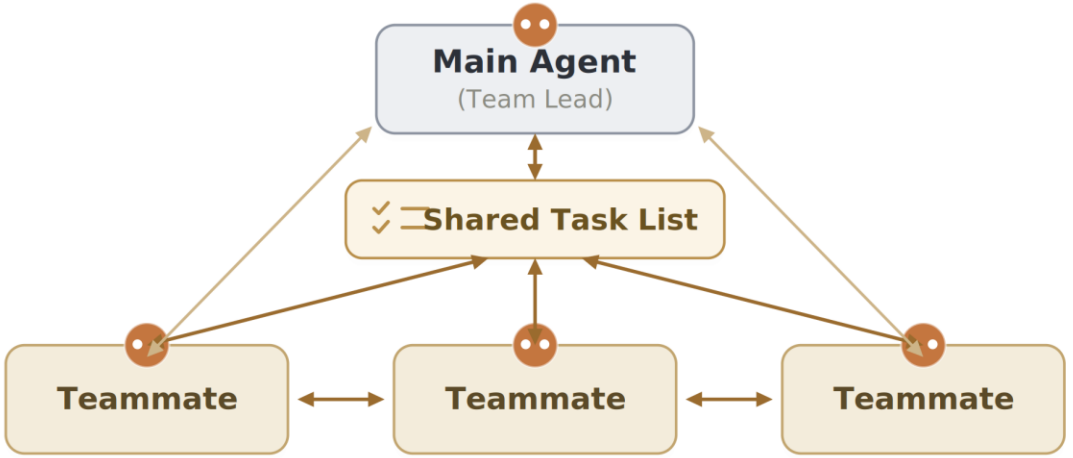


Sub-agents work in isolation and never talk to each other

- **Context** Own context window; results return to the caller.
- **Communication** Reports back to the main agent only.
- **Coordination** Main agent manages all the work.
- **Best for** Focused tasks where only the result matters.
- **Token cost** Lower — results summarized into main context.

Agent Teams

Collaboration & Independence



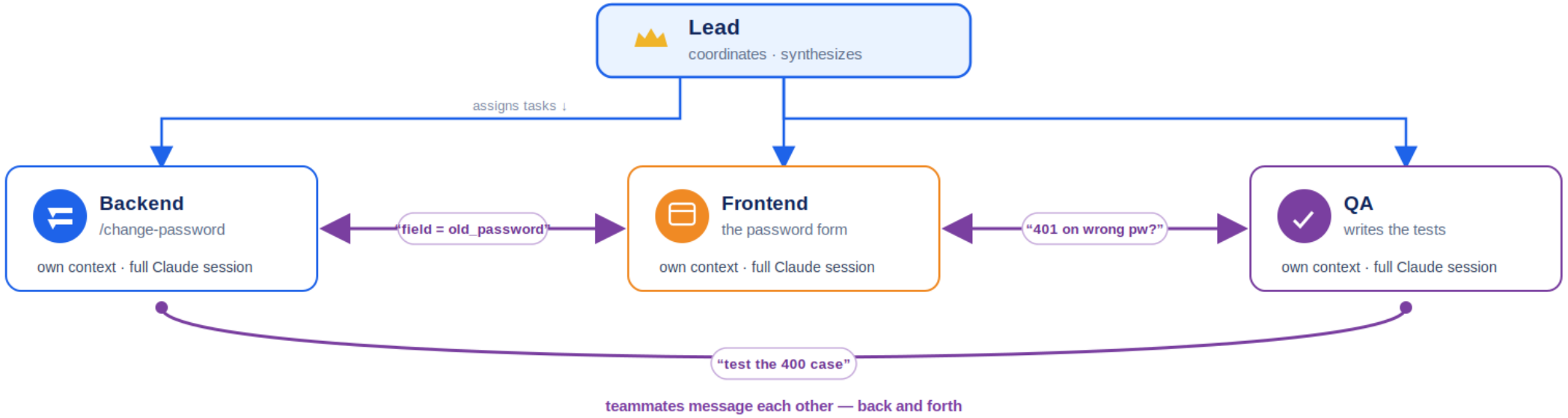
Teammates message each other directly and self-coordinate

- **Context** Own context window; fully independent instances.
- **Communication** Teammates message each other directly.
- **Coordination** Shared task list with self-coordination.
- **Best for** Complex work needing discussion & collaboration.
- **Token cost** Higher — each teammate is a separate instance.

Example: A Team of Three Subagents

The Fix — A Team That Talks

An agent team is several Claude sessions that **message each other directly** and share one to-do list.
The lead coordinates — but the teammates also talk **across**, agreeing on the contract before they build.



Shared task list

one to-do list all three claim work from

□ endpoint □ form □ tests → claimed & checked off together

Team mailbox

direct peer-to-peer messages between teammates

no single boss relays — they self-coordinate

Subagents report up. Teammates coordinate across.

Because they agree on `old_password` up front, the form, the endpoint, and the tests all fit — first try.

How the Team Coordinates?

How the Team Coordinates — Task List + Mailbox

Two shared mechanisms let the teammates self-organize: **one to-do list** they all pull from, and a **mailbox** for direct messages. Together they replace the lead having to relay everything by hand.

Shared Task List

one to-do list · every teammate claims and checks off its own work

- **Build /change-password endpoint** — owner: Backend — done
- **Add Change Password form to dashboard** — owner: Frontend — in progress
- **Write tests: 401 / 400 / success** — owner: QA — pending

How it works

- The lead populates the list from your prompt.
- Each teammate **claims** a task, works it, and marks it done.
- Everyone sees the same live status — no duplicated work.

`Ctrl+T to view`

Team Mailbox

direct peer-to-peer messages — teammates talk without the lead relaying

Backend
Endpoint expects `old_password` & `new_password`.

Frontend
Got it — matching those field names. Status on wrong pw?

Backend
`401` on wrong current password, `400` on mismatch.

QA
Perfect — writing tests for `401`, `400`, and success.

Contract agreed before anyone wrote code.
All three now use the same field names and status codes — so the form, the endpoint, and the tests fit together the first time.

The task list divides the work; the mailbox keeps them in sync.

This is what subagents can't do — the coordination happens **between the teammates**, not through you.

Turn ON the AGENT_TEAMS Feature

Turn It On & Use It (it's experimental)

Agent teams are off by default. One line in settings switches them on — then you just ask for a team in plain English.

1 · Enable the flag

`~/.claude/settings.json`

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Needs a capable model
use Opus for the lead and teammates · restart Claude Code after saving

It's a research preview
expect a few rough edges · no flag = no team (Claude won't spawn teammates)

2 · Ask for a team (plain English)

```
> Create an agent team to build a Change Password
  feature. Spawn 3 teammates:
  • Backend    — the /change-password endpoint
  • Frontend   — the form on the dashboard
  • QA         — the tests

  Have them agree on the API contract
  (fields + status codes) by messaging each
  other before writing any code.
```

The "agree on the contract first" line is the key —
it's what makes the three agents message back and forth instead of guessing.
Watch the mailbox: field names and status codes get settled between them, live.

While it runs

`Shift+↑ / ↓` cycle through teammates to watch each one

`Ctrl+T` see the shared task list (pending · in-progress · done)

Select a teammate and type to message it directly — no need to go through the lead.

Flag on → restart → ask for a team. No setup command.
The lead spawns the teammates, populates the task list, and they coordinate through the mailbox on their own.

Architecture Comparison

Decision Framework

Overview: Choose the simplest architecture that meets your communication, parallelism, and coordination requirements.

Feature	Subagent	Background Agent	Git Worktree	Agent Team
Context	Own isolated context	Independent Claude session	Separate Git worktree	Independent teammate sessions
Independent Parallel Sessions	✗	✓	✓ (Manual)	✓
Communication	Reports to parent session	Human supervises	Manual coordination	Peer-to-peer collaboration
Best For	Focused side tasks	Independent parallel work	Manual parallel development	Coordinated collaboration
Relative Usage	Lowest	Moderate	Lowest	Highest
Watch Out For	Over-delegation	Higher Claude Code usage	Manual coordination	Coordination overhead

Subagent

Examples

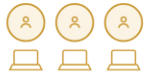
- Security review
- API documentation
- Code search



Background Agent

Examples

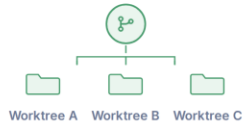
- Bug fix
- PR review
- Test investigation



Git Worktree

Examples

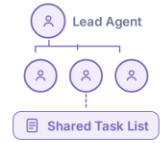
- Separate feature branch
- Experimental refactor
- Independent development



Agent Team

Examples

- Full-stack feature
- Multi-component architecture
- Parallel research



Recommendation

Start simple. Scale collaboration only when necessary.

Use Subagents for focused work, Background Agents for independent parallel execution, Git Worktrees for isolated code changes, and Agent Teams when multiple AI workers must communicate and coordinate on the same project.

Hands-on Demo: Agent Teams



Attack Surface (From Web to Agents)

The Evolving Landscape of Application Security

Instructor: Muhammad Arif Butt, PhD

The OWASP Top-10 Family

01 — FOUNDATIONAL APPLICATION SECURITY

2003 – 2025

A01-A10

OWASP TOP 10

Web Applications

The flagship list. Broken access control, injection, cryptographic & supply-chain failures in web apps.

INTRODUCED	LATEST
2003	2025

M1-M10

OWASP MOBILE TOP 10

Mobile Apps

Risks for iOS / Android apps: improper credential use, insecure storage, weak auth, supply chain.

INTRODUCED	LATEST
2014	2024

API1-API10

OWASP API SECURITY TOP 10

APIs

API-specific risks: broken object-level authorization (BOLA), broken auth, unsafe consumption.

INTRODUCED	LATEST
2019	2023

02 — THE AI-NATIVE ERA

2023 – 2026

LLM01-10

OWASP TOP 10 FOR LLMS

LLM Applications

Generative-AI risks: prompt injection, sensitive-info disclosure, data / model poisoning, excessive agency.

INTRODUCED	LATEST
2023	2025

MCP01-10

OWASP MCP TOP 10

Model Context Protocol

Securing the agent–tool protocol: tool poisoning, token leakage, shadow servers, excessive scope.

INTRODUCED	LATEST
2025	2025 β

ASI01-10

OWASP TOP 10 FOR AGENTIC APPS

Autonomous Agents

Acting AI systems: goal hijacking, tool misuse, identity / delegation gaps, rogue & cascading agents.

INTRODUCED	LATEST
2026	2026

AST01-10

OWASP AGENTIC SKILLS TOP 10

Agent Skills

The agent "behavior layer": malicious & over-privileged skills, supply-chain compromise, the lethal trifecta.

INTRODUCED	LATEST
2026	2026 β

https://owasp.org/

OWASP Top 10 for LLM Applications

LLM01:2025

Prompt Injection

RISK

Crafted inputs, direct or via retrieved/external data, override instructions to leak data, bypass safety, or misuse connected tools.

MITIGATION

Dual-LLM verification (watchdog pattern) screens inputs/outputs for injection; enforce input/output separation and least-privilege tool access.

LLM02:2025

Sensitive Information Disclosure

RISK

Model reveals PII, secrets, API keys, or proprietary/training data through outputs, logs, or retrieval context.

MITIGATION

DLP scanning on output buffers redacts secrets & PII before responses; scrub logs and return generic errors, no verbose stack traces.

LLM03:2025

Supply Chain

RISK

Compromised base models, datasets, adapters, or libraries introduce backdoors and hidden flaws.

MITIGATION

Verified sources + cryptographic verification (SafeTensors, SHA-256, Sigstore/Cosign signing); audit dependencies with `pip-audit` / `safety`.

LLM04:2025

Data & Model Poisoning

RISK

Tampered training, fine-tuning, or RAG data corrupts model behavior, accuracy, and safety.

MITIGATION

Vet data provenance; anomaly-detect training data; integrity checks (checksums, signed artifacts); admission controllers verify signatures before load.

LLM05:2025

Improper Output Handling

RISK

Downstream systems trust unvalidated output, enabling XSS, SSRF, or remote code execution.

MITIGATION

Output sanitization: encode/allow-list HTML (e.g. `bleach`); never feed raw output into SQL/shell; use parameterized queries and sandboxed rendering.

LLM06:2025

Excessive Agency

RISK

Too much functionality, permission, or autonomy lets the model take harmful, unintended actions.

MITIGATION

Tool-definition manifests (allow-listed tools, scoped permissions, rate limits); least privilege + ephemeral credentials; HITL approval for high-impact actions.

LLM07:2025

System Prompt Leakage

RISK

Internal prompts, policies, or tool schemas are exposed, revealing secrets and weakening controls.

MITIGATION

Keep secrets/credentials out of prompts; enforce authorization and guardrails outside the prompt; filter outputs to detect prompt disclosure.

LLM08:2025

Vector & Embedding Weaknesses

RISK

RAG retrieval and embeddings introduce poisoning, leakage, and cross-tenant data exposure.

MITIGATION

Tenant isolation / namespacing in the vector store; permission-aware retrieval; validate & sanitize documents before embedding; monitor anomalous retrievals.

LLM09:2025

Misinformation

RISK

Confident but false or fabricated output misleads users and propagates into downstream decisions.

MITIGATION

Ground answers in trusted sources (RAG) with citations; cross-verify with a watchdog model; human oversight for high-stakes outputs; surface uncertainty.

LLM10:2025

Unbounded Consumption

RISK

Uncontrolled resource use enables denial-of-service, runaway cost (token inflation / EDoS), and model theft.

MITIGATION

Rate limiting & resource controls (token-bucket, per-tenant quotas, max-token caps); token-inflation anomaly detection; throttle or queue expensive requests.

Core defenses

- **Least privilege** — scope every token, tool & skill to the minimum
- **Verify & isolate** — sign, sandbox & validate untrusted inputs and outputs
- **Human-in-the-loop** — require approval for high-impact, irreversible actions
- **Observe everything** — log, monitor & audit for anomalies
- **Govern the inventory** — know what's connected, updated & running

https://genai.owasp.org/llm-top-10/

OWASP Top 10 for MCP

MCP01:2025

Token Mismanagement & Secret Exposure

RISK

Hard-coded or long-lived tokens leak through logs, config, or context spillover, exposing every connected system.

MITIGATION

Short-lived, scoped OAuth 2.1 tokens; continuous secrets detection across repos & CI; guardrails that catch secrets before they leave.

MCP02:2025

Privilege Escalation via Scope Creep

RISK

Permissions that start broad or grow over time let agents, or attackers, act far beyond intent across trust boundaries.

MITIGATION

Least-privilege scopes; automated scope expiry; periodic access reviews; runtime policy blocking sensitive-to-public data movement.

MCP03:2025

Tool Poisoning

RISK

Malicious instructions hidden in a tool's description, schema, or output (rug pulls, schema poisoning, tool shadowing) trigger harmful actions.

MITIGATION

Signed, version-pinned tools; scan tool descriptions at install; alert on any post-approval description drift.

MCP04:2025

Software Supply Chain & Dependency Tampering

RISK

Compromised packages, connectors, or dependencies add backdoors or silently change behavior (e.g. CVE-2025-6514 in mcp-remote, CVSS 9.6).

MITIGATION

Signed components & verified publishers; maintain an AIBOM with provenance; run SCA before agents pull new tools.

MCP05:2025

Command Injection & Execution

RISK

Untrusted input in shell commands, scripts, or code lets attackers run arbitrary commands, the most common MCP CVE class.

MITIGATION

Parameterized command APIs (avoid shell/eval); strict input validation; deny-by-default egress; sandbox servers in containers.

MCP06:2025

Intent Flow Subversion

RISK

Malicious context acts as a hidden instruction channel (indirect prompt injection), steering the agent off the user's goal.

MITIGATION

Treat all retrieved content as untrusted; isolate context and quarantine instructions; require human-in-the-loop approval for sensitive actions.

MCP07:2025

Insufficient Authentication & Authorization

RISK

Weak or missing identity checks let the wrong users or services reach tools, data, and actions; exposed endpoints become open doors.

MITIGATION

OAuth 2.1 with per-server audience validation; MFA for human approval of sensitive actions; managed identities for service-to-service calls.

MCP08:2025

Lack of Audit & Telemetry

RISK

Without detailed, tamper-proof logs, attacks and abnormal behavior are hard to detect or stop, the meta-risk hiding all others.

MITIGATION

Immutable audit trails of tool calls & context changes; real-time anomaly alerting; sequence-level behavioral monitoring.

MCP09:2025

Shadow MCP Servers

RISK

Unapproved, unmonitored MCP servers outside official oversight run with default creds and permissive configs, quietly widening the attack surface.

MITIGATION

Continuous discovery + AIBOM inventory; allowlist enforcement at IDE and CI/CD; bring shadow AI under governance.

MCP10:2025

Context Injection & Over-Sharing

RISK

Shared or persistent context windows leak sensitive data across users, tasks, or sessions (cross-tenant exposure).

MITIGATION

Scoped context windows; ephemeral memory by default; strict cross-tenant isolation enforced at the protocol layer.

Core defenses

- **Least privilege** — scope every token, tool & skill to the minimum
- **Verify & isolate** — sign, sandbox & validate untrusted inputs and outputs
- **Human-in-the-loop** — require approval for high-impact, irreversible actions
- **Observe everything** — log, monitor & audit for anomalies
- **Govern the inventory** — know what's connected, updated & running

https://owasp.org/www-project-mcp-top-10/

OWASP Top 10 for Agentic Applications



ASI01:2026

Agent Goal Hijack

RISK

Malicious instructions in documents, tools, or data redirect the agent's objectives and decision path.

MITIGATION

Separate trusted instructions from retrieved data; validate tool outputs before they steer planning; constrain the agent to a defined goal scope (least agency).

ASI02:2026

Tool Misuse & Exploitation

RISK

Agents are manipulated into chaining, looping, or over-invoking legitimate tools unsafely, causing harm despite valid permissions.

MITIGATION

Scope every tool tightly with per-tool permission boundaries; require validation on high-impact actions; limit what each tool can do rather than trusting the agent.

ASI03:2026

Agent Identity & Privilege Abuse

RISK

Delegated authority, impersonation, or cross-agent trust lets inherited or cached credentials perform unauthorized privileged actions.

MITIGATION

Least privilege per agent identity; short-lived scoped credentials over standing ones; keep every agent's permissions discoverable and auditable.

ASI04:2026

Agentic Supply Chain Compromise

RISK

Agents discover and import external tools, schemas, models, prompt templates, or personas at runtime without verification.

MITIGATION

Vet and pin agent dependencies; scan tool and skill registries before install; verify the provenance of prompt templates, personas, and models.

ASI05:2026

Unexpected Code Execution

RISK

Agent-generated or agent-triggered code runs without sufficient validation, sandboxing, or isolation (RCE, sandbox escape, exfiltration).

MITIGATION

Run generated code in a strict sandbox; deny network egress by default; require explicit approval before any generated command touches a real system.

ASI06:2026

Memory & Context Poisoning

RISK

Injected or corrupted persistent memory, embeddings, or RAG stores silently shape the agent's future reasoning and actions.

MITIGATION

Validate what enters long-term memory; segment memory by trust level; periodically reconcile stored context against a trusted source.

ASI07:2026

Insecure Inter-Agent Communication

RISK

Messages between agents are intercepted, spoofed, replayed, or injected, manipulating planners and executors and forging false consensus.

MITIGATION

Authenticate and sign every agent-to-agent message; verify message integrity with mutual authentication; reject unverifiable consensus.

ASI08:2026

Cascading Agent Failures

RISK

A single error or hallucination propagates through connected tools and agents, amplifying into large-scale impact.

MITIGATION

Add circuit breakers between agents; validate outputs at each handoff; cap the depth of autonomous chains so one fault cannot cascade unchecked.

ASI09:2026

Human-Agent Trust Exploitation

RISK

Fluent, authoritative output exploits human over-reliance into approving harmful actions or sharing sensitive data.

MITIGATION

Separate the agent's conversation from the security boundary; route consent and high-impact approvals through out-of-band confirmation, never the chat alone.

ASI10:2026

Rogue Agents

RISK

Goal drift, collusion, reward hacking, or runaway autonomy pushes agents beyond their limits while still appearing legitimate.

MITIGATION

Monitor agent behavior against expected baselines; enforce strong identity on every agent; isolate orchestration so one compromised agent cannot assume trusted roles.

Core defenses

- **Least privilege** — scope every token, tool & skill to the minimum
- **Verify & isolate** — sign, sandbox & validate untrusted inputs and outputs
- **Human-in-the-loop** — require approval for high-impact, irreversible actions
- **Observe everything** — log, monitor & audit for anomalies
- **Govern the inventory** — know what's connected, updated & running

https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

OWASP Top 10 for Agentic Skills



AST01:2026 Malicious Skills

RISK

Skills built to steal data, run hidden commands, or take actions you never approved, executing with the agent's full permissions.

MITIGATION

Require ed25519 signatures + Merkle-root registry signing; behavioral scanning at publish and install; never auto-run "Prerequisites"; hash-pin and alert on changes.

AST02:2026 Supply Chain Compromise

RISK

Hacked dependencies, registries, or download channels swap in a tampered skill; config files quietly become execution paths.

MITIGATION

Provenance tied to a verified signing identity; registry transparency logs; pin nested deps to sha256 hashes; treat config files (hooks, settings) as code behind trust gates.

AST03:2026 Over-Privileged Skills

RISK

A skill granted more access than it needs magnifies the damage if it is compromised or hit by prompt injection.

MITIGATION

Require a least-privilege permission manifest; per-skill scoped credentials; domain allowlists over on/off network; flag writes to identity files; enforce at runtime.

AST04:2026 Insecure Metadata

RISK

Missing, inaccurate, or faked metadata makes a skill look safer or different than it is (brand impersonation, understated permissions).

MITIGATION

Static analysis + manifest linting at publish; validate declared permissions against runtime behavior; scan prose for ASCII-smuggling/base64; trademark checks; surface provenance.

AST05:2026 Unsafe Deserialization

RISK

Loading config files (YAML/JSON) without safe parsing lets hidden payloads run code, often on skill load before any user action.

MITIGATION

Safe loaders with dangerous tags disabled (yaml.safe_load); parse and validate configs in a sandbox against a schema; treat bundled deps as untrusted; drop privileges before parsing.

AST06:2026 Weak Isolation

RISK

Skills run on the host instead of a sandbox, reaching files and credentials they shouldn't.

MITIGATION

Default container/Docker isolation with host-mode opt-in; per-skill process isolation; seccomp/AppArmor; bind control interfaces to authenticated localhost; rate-limit WebSockets.

AST07:2026 Update Drift

RISK

Without pinned versions, a safe skill can quietly turn malicious in an update, or leave known CVEs unpatched.

MITIGATION

Pin to immutable sha256 hashes; verify signatures on every update; freeze / no hot-reload in production; CVE advisory alerts; human approval; keep a versioned skill inventory.

AST08:2026 Poor Scanning

RISK

Pattern-only scanners miss threats hidden in plain-language instructions.

MITIGATION

Behavioral/semantic scanning of intent, not just signatures; scan code and instruction layers separately; sandbox and compare declared vs actual behavior; re-scan over time.

AST09:2026 No Governance

RISK

No inventory, auditing, or policy means teams can't see what runs or respond when one misbehaves.

MITIGATION

Central skill inventory (name, version, hash, installer); install-approval workflow; scoped agentic identities; audit-log every skill action; CMDB/CASB integration; revocation process.

AST10:2026 Cross-Platform Reuse

RISK

A skill safe on one platform can be dangerous on another when security settings are lost in porting.

MITIGATION

Adopt a Universal Skill Format; re-validate security metadata on every port; normalize risk-tier, permission & signature fields; share cross-registry threat intel; platform-agnostic scanners.

Core defenses

- **Least privilege** — scope every token, tool & skill to the minimum
- **Verify & isolate** — sign, sandbox & validate untrusted inputs and outputs
- **Human-in-the-loop** — require approval for high-impact, irreversible actions
- **Observe everything** — log, monitor & audit for anomalies
- **Govern the inventory** — know what's connected, updated & running

Q/A



Thank you!